SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS **COURSE CODE:** CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools.

(BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I **M.Greeshma/192524195**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**M.Greeshma**

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

Sol:

```
Algorithm NetworkPerformanceAnalysis
   Input: total_bits_received, time_seconds, packets_sent, packets_received
   Output: throughput_bps, packet_loss_percentage, classification, summary

   Begin
      // Step 1: Validate inputs
      If time_seconds = 0 Then
         Display "Error: Time cannot be zero."
         Exit
      EndIf
```

```
If packets_sent = 0 Then
   Display "Error: Packets sent cannot be zero."
   Exit
EndIf

// Step 2: Calculate throughput
throughput_bps ← total_bits_received / time_seconds

// Step 3: Calculate packet loss percentage
lost_packets ← packets_sent - packets_received
packet_loss_percentage ← (lost_packets / packets_sent) * 100

// Step 4: Classify network performance
If throughput_bps > 1500 AND packet_loss_percentage < 10 Then
   classification ← "Efficient"
Else
   classification ← "Inefficient"
EndIf

// Step 5: Display results
Display "Network Throughput: ", throughput_bps, " bps"
Display "Packet Loss: ", packet_loss_percentage, "%"
Display "Network Classification: ", classification

// Step 6: Output summary
summary ← "Summary: The network achieved a throughput of " + throughput_bps +
        " bps with a packet loss of " + packet_loss_percentage + "%. " +
```

"Overall, the network is classified as " + classification + "."

Display summary

End

Applying the given values:

- **Total bits received:** 8000
- **Time:** 4 seconds
- **Packets sent:** 200
- **Packets received:** 180

Throughput:

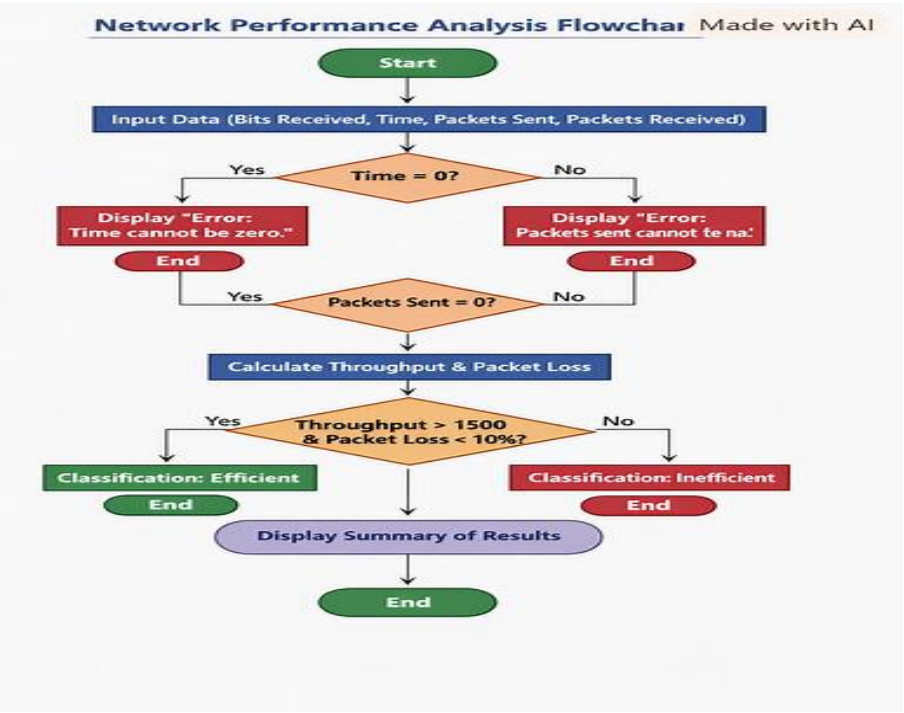
$$\frac{8000}{4} = 2000 \text{ bps}$$

Packet Loss Percentage:

$$\frac{200 - 180}{200} \times 100 = 10\%$$

Classification:

- Throughput = 2000 bps (> 1500) ✓
- Packet loss = 10% (not < 10%) ✗ → **Network is Inefficient**



2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

Sol:

```
Algorithm BandwidthMonitoring
   Input: num_links, bandwidth_usage[num_links]
   Output: status_report

   Begin
      // Step 1: Initialize monitoring
      Display "Starting bandwidth monitoring..."

      // Step 2: Continuous monitoring loop
      While True Do
         For i ← 1 To num_links Do
            // Step 3: Collect bandwidth usage for each link
            bandwidth_usage[i] ← GetBandwidthUsage(i)

            // Step 4: Check for overloaded links
            If bandwidth_usage[i] > 80 Then
               Display "Alert: Link ", i, " utilization = ", bandwidth_usage[i], "%"
```

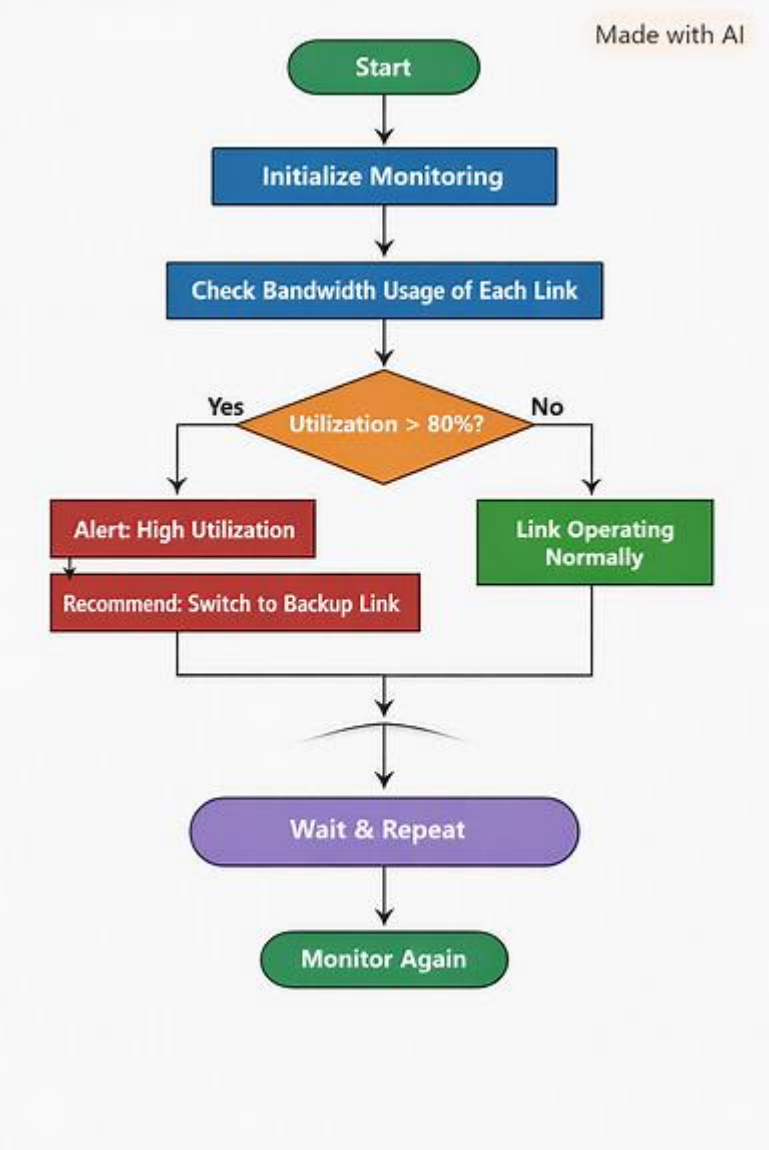
```
            Display "Recommendation: Switch traffic to backup link."
        Else
            Display "Link ", i, " utilization = ", bandwidth_usage[i], "% (Normal)"
        EndIf
    EndFor

    // Step 5: Wait before next monitoring cycle
    Wait(Interval_Time)  // e.g., 5 seconds
  EndWhile
End
```

Explanation

- The algorithm **periodically collects bandwidth usage** for each network link using a loop.
- It **stores usage values in an array** (bandwidth_usage) for easy tracking and comparison.
- When any link exceeds **80% utilization**, it triggers an **alert** and recommends switching traffic to a backup link.
- The continuous loop ensures **real-time monitoring**, helping administrators respond quickly to congestion.

⚙ **How It Supports Efficient Bandwidth Management**

Feature	Benefit
Periodic data collection	Keeps utilization data up to date
Overload detection	Prevents bottlenecks and packet delays
Backup link recommendation	Ensures uninterrupted connectivity
Continuous monitoring	Enables proactive congestion control



3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the

monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

Sol:

```
Algorithm AdaptivePathSelection
  Input: num_links, bandwidth[num_links], delay[num_links], utilization[num_links]
  Output: best_path, alert_message

  Begin
    Display "Starting network path monitoring..."

    While True Do
      For i ← 1 To num_links Do
        // Step 1: Collect metrics for each link
        bandwidth[i] ← GetBandwidth(i)
        delay[i] ← GetDelay(i)
        utilization[i] ← GetUtilization(i)

        // Step 2: Calculate link quality score
        // Higher bandwidth and lower delay/utilization yield better score
        quality_score[i] ← (bandwidth[i] / (delay[i] * utilization[i]))
```

```
        // Step 3: Detect overloaded or failed links
        If utilization[i] > 90 OR bandwidth[i] = 0 Then
            Display "Alert: Link ", i, " overloaded or failed!"
            Display "Switching traffic to alternate path..."
            alert_message ← "Link " + i + " requires attention."
        EndIf
    EndFor

    // Step 4: Select best path based on highest quality score
    best_path ← IndexOf(Max(quality_score))
    Display "Best communication path: Link ", best_path

    // Step 5: Wait before next monitoring cycle
    Wait(Interval_Time)  // e.g., 10 seconds
  EndWhile
End
```

⚙ How It Improves Network Reliability and Efficiency

Feature	Benefit
Dynamic metric collection	Adapts to real-time changes in bandwidth, delay, and utilization
Quality-based path selection	Chooses the most efficient route, not just the shortest
Overload/failure detection	Automatically reroutes traffic to maintain connectivity
Periodic monitoring	Ensures continuous optimization and fault tolerance

Adaptive Network Path Selection

In a dynamic network environment, link conditions such as **bandwidth**, **delay**, and **utilization** fluctuate throughout the day. To maintain optimal communication between headquarters and branch offices, the administrator must continuously evaluate these parameters and select the best-performing path.

◈ Algorithm Concept

1. **Data Collection:** The algorithm periodically gathers bandwidth, delay, and utilization values for each link and stores them in arrays or lists.
2. **Quality Score Calculation:** Each link's performance is evaluated using the formula:

$$\text{Quality Score} = \frac{\text{Bandwidth}}{\text{Delay} \times \text{Utilization}}$$

- High bandwidth increases the score.
- High delay or utilization decreases the score.

3. **Path Selection:** The link with the **highest quality score** is chosen as the best communication path.
4. **Fault Detection:** If a link's utilization exceeds 90% or bandwidth drops to zero, the algorithm triggers an **alert** and automatically switches traffic to an alternate path.
5. **Continuous Monitoring:** The process repeats at regular intervals to adapt to changing network conditions.